Securing RAG from Prompt Injection & Embedding Inversion Attack

Phase II - Second Review

Submitted By:

2022BCY0041 Sandra Sabu
2022BCY0027 Paranji Punith Kumar
2022BCY0060 Mounika Seera
2022BCY0017 Kottagorla Sai Manoj

Guided By,
Dr. Kanchan Lata Kashyap
Assistant Professor, IIIT Kottayam

Introduction

What is Retrieval-Augmented Generation (RAG)?

- A framework that enhances Large Language Models (LLMs) by retrieving relevant information from external knowledge sources before generating a response.

Core Problem: Mitigates LLM **hallucinations** and provides **up-to-date, traceable** information.

Why it's important:

- **Accuracy:** Reduces hallucinations and provides factual, current answers.
- **Transparency:** Provides sources/citations for answers.
- **Efficiency:** Cheaper and faster than constantly re-training LLMs on new data.

Why Secure RAG for Financial Services?

- Handles sensitive financial data (account numbers, transactions, personal information)
- Regulatory compliance requirements
- High-value target for attackers due to financial incentives

Phase I Summary

1. Core RAG Implementation

- Document ingestion (PDF, DOCX, TXT)
- Embedding generation via Sentence-Transformers
- FAISS(Facebook AI Similarity Search) based vector indexing (Vector DB)
- LLM integration

2. Security Framework Integration

- Semantic safety scoring (NLI models)
- Query rewriting for unsafe inputs
- Safe context aggregation & filtering
- Output validation and blocking

Literature Review

Title	Author	Methodology	Results	Limitations
Truthful Text Sanitization Guided by Inference Attacks[1]	**2025 — Ildikó Pilán, Benoit Marnane-Sztobó, David Sánchez**	Proposes replacement of candidates for disclosive terms with different levels of generalization. and selecting the most appropriate replacement among those candidates and inference-attack simulation to sanitize sensitive text while preserving semantic meaning and utility.	Achieves strong privacy protection, reduces sensitive attribute inference, maintains high readability and semantic fidelity, and outperforms traditional anonymization methods.	Computational overhead; depends on LLM quality and prompt design; mainly text-focused; scalability and real-time deployment challenges.
User Identification of Pseudonyms without Identity Information Exposure in Access Federations [2]	**Hiroyuki Sato, Yasuo Okabe, Motonori Nakamura**	Proposes a privacy-preserving user identification framework using encrypted pseudonyms and a trusted counting server in federated systems.	Enables secure user-level access control and tracking without revealing real identity; supports privacy-preserving authentication.	Adds system complexity and communication overhead; depends on trusted infrastructure; scalability issues.

Literature Review

Title	Year & Author	Methodology	Results	Limitations
RAG Powered LLMs for QA: Evolution, Challenges, Applications, and Future Directions [4]	**2025 Hafiz Muhammad Ali Zeeshan et al.**	Involves synthesizing and analyzing existing literature on RAG architectures (Naive, Advanced, Modular), their core components **(Retrieval, Generation, Augmentation)**, applications, and evaluation benchmarks.	The evolution of RAG, highlighting its effectiveness in improving QA tasks across various domains by reducing hallucinations and incorporating external knowledge.	Its limitations are the inherent challenges of RAG it discusses, such as sensitivity to noisy retrieved data and the computational complexity of advanced architectures.
Task decomposition and RAG as Design Patterns for LLM-based Systems [5]	**2025 Orlando Marquez Ayala**	Proposes design patterns based on the author's experience building a real-world enterprise application using Task Decomposition and RAG to address LLM challenges like hallucination and latency.	It formalizes Task Decomposition and RAG as design patterns, highlighting benefits like improved correctness, user engagement, modularity, testability, and easier debugging for LLM-based systems.	The patterns are derived from a single use case, which may limit generalizability. The approach also introduces overhead from task orchestration and increased LLM calls, potentially reducing scalability

Literature Review

Title	Year & Author	Methodology	Results	Limitations
Backdoored Retrievers for Prompt Injection Attacks on RAG[6]	**2024 — Cody Clop, Yannick Teglia**	Investigates vulnerabilities in Retrieval-Augmented Generation (RAG) systems to prompt injection. Compares **corpus poisoning** vs **backdoor attacks** on retrievers using Llama-3, Vicuna, and Mistral. Evaluated with BEIR datasets.	Backdoor attacks achieved nearly **100% success rate** in retrieving poisoned documents; corpus poisoning was effective in narrow domains (e.g., Alzheimer's) but weaker in broad domains (nutrition). Vicuna showed highest robustness, Llama-3 most vulnerable.	Backdoor attack requires victim to fine-tune on attacker's dataset (less realistic). Corpus poisoning less effective for broad/open domains. Did not evaluate defenses in depth.
SoK: Prompt Hacking of Large Language Models[7]	**2024 — Baha Rababah et al.**	Systematic overview of **prompt hacking**: jailbreaking, prompt injection, and prompt leaking. Proposes a new classification framework with **five categories of LLM responses** instead of binary safe/unsafe. Surveys attacks and defenses.	Provides taxonomy of attacks and defenses, highlighting weaknesses of current mitigation (filtering, adversarial training, red teaming, etc.). Demonstrates that LLMs remain vulnerable to carefully crafted jailbreaks and injections.	Primarily survey-based; limited original experiments. Focus mostly on GPT-3.5; lacks evaluation across wider range of LLMs.
Forensic Analysis of Indirect Prompt Injection Attacks on LLM Agents[8]	**2024 — Maxim Cherny shev et al.**	Introduces a forensic analysis framework for detecting **Indirect Prompt Injection (IPI)** in LLM agent execution logs. Uses **AgentDojo benchmark** (97 tasks, 629 security test cases). Evaluated 12 LLMs (Claude, GPT-4o, Gemini, LLaMA, Mistral, etc.).	Proposed forensic log analysis method shows **promising detection capability**. Some models (Claude-3 Opus, GPT-4o) performed consistently; others varied widely by test suite. Gemini-1.5-Flash had perfect scores in some cases.	Results preliminary; variability across models/test suites; context window limitations affect analysis. Requires further validation with real-world forensic cases.

Literature Review

Title	Year & Author	Methodology	Results	Limitations
Transferable Embedding Inversion Attack: Uncovering Privacy Risks in Text Embeddings without Model Queries [9]	**2024 National Taiwan University**	Proposed a transferable embedding inversion attack using a surrogate encoder with consistency regularization, adversarial training, and a GPT-based decoder to reconstruct text from embeddings without model queries.	Outperformed direct attacks by 40–50% , achieved 98–99% recovery of sensitive attributes (age, sex) in the MIMIC-III clinical dataset; effective even with limited or out-of-domain data.	Performance drops for longer sentences, depends on availability of leaked embedding–text pairs, and reconstructions remain imperfect compared to oracle access.
Information Leakage in Embedding Models [10]	**2020 — Congzheng Song & Ananth Raghunathan**	Investigated **embedding-level attacks**: embedding inversion (reconstructing text from vectors), attribute inference (guessing hidden properties), and membership inference (detecting if a record was in training data). Evaluated these attacks on NLP datasets and tested **defenses** like adversarial training and regularization to harden models.	Embeddings **do leak sensitive information** such as actual words, authorship, and training membership with high precision/recall. Demonstrated that **adversarial training** and noise injection reduce leakage while maintaining most model utility. Highlighted that even common embedding models are vulnerable when used in downstream systems.	Defenses come with a **trade-off**: adding noise or adversarial training reduces leakage but may **degrade accuracy or retrieval quality**. Attacks remain **effective on many modern models**, showing current mitigations are not comprehensive. More research needed to balance **utility vs. privacy** and to design robust privacy-preserving embedding methods.

Motivation

Financial institutions use RAG for analytics and reporting.

However, RAG systems are **vulnerable to attacks** such as:

- **Prompt Injection** - malicious instructions
- **Embedding Inversion** - reconstruction of sensitive data

Motivation:

Develop a **secure RAG pipeline** that maintains both **accuracy** and **confidentiality**.

Problem Statement

- RAG systems in financial services face security risks like prompt injection and embedding inversion attacks, which can leak sensitive data or manipulate outputs.

- Existing defenses are limited and often reduce performance, highlighting the need for a robust Secure RAG framework that ensures security without compromising accuracy and efficiency.

Our Project Objectives:

1. Develop a **functional RAG system**.

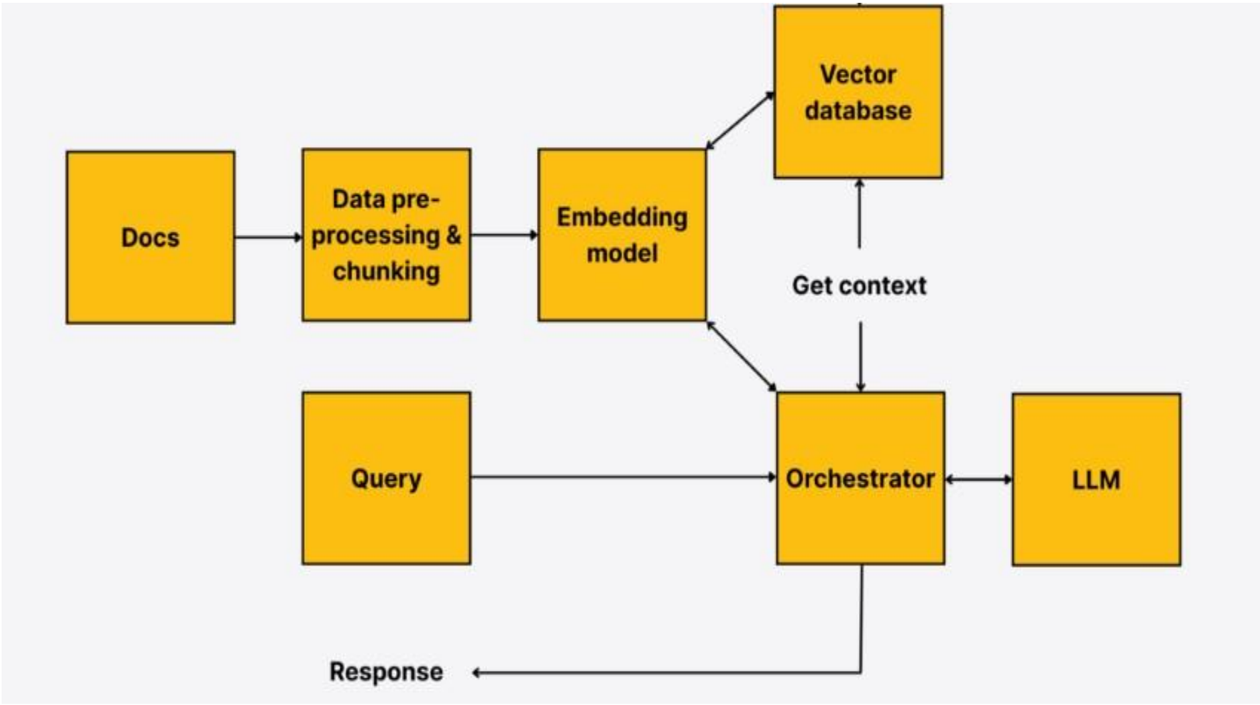
2. Integrate **security mechanisms** (sanitization, encryption, safety scoring, guard models).

3. Ensure **safe retrieval and generation**.

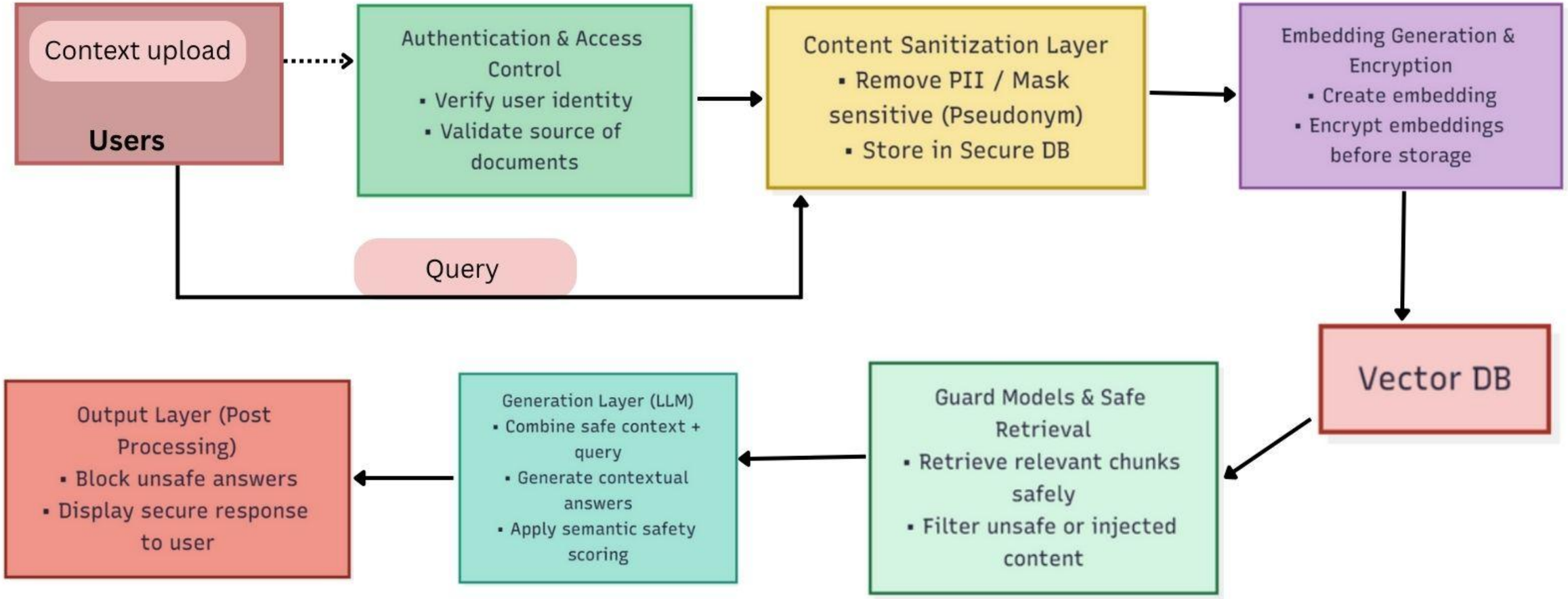
Architecture

RAG Architecture Process:

1. **Data Ingestion** – Preprocess documents and create embeddings.
2. **Retriever** – Finds relevant information from a database.
3. **Generator** – Produces responses using retrieved context.
4. **Post-processing** – Cleans and formats the final output.



Proposed Secure Architecture



Phase 2 - Review 1 Summary

- Implemented Authentication Module using JWT & RBAC
  → Ensures secure login and controlled access.

- Developed Content Sanitization Layer
  → Detects and masks PII (sensitive data) using regex & pseudonymization.

- Designed secure workflows for authentication and sanitization
  → Improves data privacy, compliance, and traceability.

Layer 3: Embedding & Encrypted Storage

Purpose:

- Protect embeddings and document data from leakage & inversion attacks.
- Ensure secure storage of sensitive information.

Key Functionalities

Embedding Generation:

- Uses Sentence-Transformers to create vector embeddings.

Encrypted Storage:

- Embeddings are encrypted before storage
- Chunk text is also encrypted at rest

Controlled Decryption:

- Data is decrypted only during secure retrieval

Technology Used:

- Fernet (Cryptography library) for encryption/decryption
- FAISS for vector storage

Security Benefits:

- Prevents embedding inversion attacks.
- Protects against data leakage from storage.
- Ensures confidentiality of both text and embeddings.

Limitations:

- Adds latency during encryption/decryption
- Requires secure key management.

Workflow:

- Step 1 – Text Chunking
- Step 2 – Embedding Generation
- Step 3 – Encryption (Embeddings + Text)
- Step 4 – Storage in FAISS
- Step 5 – Retrieval
- Step 6 – Decryption (Controlled Access)

“In our implementation, both embeddings and chunk text are encrypted using Fernet, ensuring protection against embedding inversion and storage-based attacks.”

Benefits of Phase 2

- Strong access control
- Data confidentiality
- Reduced legal risks
- Secure preprocessing pipeline

Conclusion

- **Developed a complete Secure RAG system with a 7-layer architecture**
- **Integrated security across all stages:**
  → Authentication, Sanitization, Encryption, Retrieval, Generation, Validation
- **Successfully:**
  - Prevents prompt injection attacks.
  - Reduces data leakage & embedding inversion risks.
  - Ensures secure and trustworthy responses.
- **Maintains RAG benefits:**
  - Accurate responses.
  - Context-aware generation.
  - Efficient knowledge retrieval.

"Built a secure, reliable, and deployable RAG framework suitable for financial and sensitive applications."

References

[1] I. Pilán, B. Marnane-Sztobó, and D. Sánchez, “Truthful text sanitization guided by inference attacks,” *Applied Soft Computing*, vol. 165, p. 111013, 2025.

[2] H. Sato, Y. Okabe, and M. Nakamura, “User Identification of Pseudonyms without Identity Information Exposure in Access Federations,” *IEICE Transactions on Information and Systems*, vol. E99-D, no. 10, pp. 2568–2577, Oct. 2016.

[3] Rathod, Vishal, et al. "Privacy and security challenges in large language models." *2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC)*. IEEE, 2025.

[4]Zeeshan, Hafiz Muhammad Ali, et al. "RAG Powered LLMs for QA: Evolution, Challenges, Applications, and Future Directions." *2025 International Conference on Communication Technologies (ComTech)*. IEEE, 2025.

[5] Ayala, Orlando Marquez. "Task Decomposition and RAG as Design Patterns for LLM-Based Systems." *2025 IEEE/ACM 4th International Conference on AI Engineering–Software Engineering for AI (CAIN)*. IEEE, 2025.

[6] C. Clop and Y. Teglia, *“Backdoored Retrievers for Prompt Injection Attacks on Retrieval Augmented Generation of Large Language Models,”* arXiv preprint arXiv:2410.14479, Oct. 2024.

[7] B. Rababah, S. Wu, M. Kwiatkowski, C. K. Leung, and C. G. Akcora, *“SoK: Prompt Hacking of Large Language Models,”* in *Proc. IEEE Int. Conf. Big Data (Big Data)*, 2024, pp. 5392–5400. doi: 10.1109/BigData62323.2024.10825103.

[8] M. Chernyshev, Z. Baig, and R. Doss, *“Forensic Analysis of Indirect Prompt Injection Attacks on LLM Agents,”* in *Proc. IEEE 6th Int. Conf. Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*, 2024, pp. 409–414. doi: 10.1109/TPS-ISA62245.2024.00053

[9] Huang, Yu-Hsiang, et al. "Transferable embedding inversion attack: Uncovering privacy risks in text embeddings without model queries." *arXiv preprint arXiv:2406.10280* (2024

[10] Song, Congzheng, and Ananth Raghunathan. "Information leakage in embedding models." *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*. 2020.

Thankyou !!